Hierarchical Edge AI Architecture and On-Demand Validation for Energy-Efficient Robotic Monitoring

Darlan de Castro Silva Filho
Master's Program in Electrical Engineering (PPGEE)
Federal University of Piaui, Brazil
Teresina, Brazil
darlan.filho@ufpi.edu.br

Joao Pedro Gomes Carvalho
Master's Program in Computer Science (PPGCC)
Federal University of Piaui, Brazil
Teresina, Brazil
j.pedrogomes2020@ufpi.edu.br

Ranulfo Bezerra
Tough Cyberphysical AI Research Center
Tohoku University
Sendai, Japan
bezerra.ranulfo@rm.is.tohoku.ac.jp

Ricardo de Andrade Lira Rabelo
Master's Program in Electrical Engineering (PPGEE)
Federal University of Piaui, Brazil
Teresina, Brazil
ricardoalr@ufpi.edu.br

***Abstract*—Modern agricultural monitoring requires timely visual inspection while operating under constraints of energy, connectivity, and operational cost. This paper proposes an event-driven hierarchical architecture for energy-efficient visual monitoring of cashew leaves. The first level uses fixed low power ESP32-S3 nodes with a camera and local TinyML inference to trigger events. The second level uses a mobile robot that is dispatched only when the fixed layer generates an alert, reducing blind periodic patrolling. The evaluation combines physical measurements of inference time and energy on the embedded node with Unity and ROS2 robotic simulation to estimate travel time and energy for on demand validation. The detection model achieved an mAP50 of 80.47%, while the classification model reached an average accuracy of 95.72% and an average F1 score of 93.47%. The fixed node consumed less than 0.61 mWh per cycle even with an alert. Compared with complete robotic patrolling, the proposed architecture reduced energy consumption by up to 83.20% with 1 alert per cycle and remained advantageous with up to 20 alerts in a 25 points grid. These results provide preliminary evidence that low power TinyML screening can reduce robotic energy expenditure in visual agricultural monitoring.**

***Index Terms*—TinyML, edge computing, smart agriculture, plant disease detection, energy-efficiency, Internet of Things**

I. Introduction

Modern agriculture faces increasing pressure to improve productivity, sustainability, and resource efficiency under rising production costs, water scarcity, climate change, and the emergence of pests and diseases [1]. In response, Agriculture 4.0 integrates Internet of Things (IoT) devices, artificial intelligence (AI), sensors, unmanned aerial vehicles, and robotics to support data-driven field management [2]. The early identification of plant symptoms is particularly important because it enables intervention before disease spreads and causes significant productivity losses. Cameras, embedded sensors, AI, and mobile robots can increase the frequency and coverage of agricultural inspection while reducing dependence on continuous manual observation. More broadly, the integration of IoT, robotics, and AI provides a path toward connected systems in which distributed devices cooperate to perceive, interpret, and act upon events in the physical environment [3].

Despite this potential, the adoption of these technologies remains slow and uneven, especially in developing countries and among smallholder farmers, because of high initial investment, limited digital literacy, poor connectivity, and privacy concerns [1]. Technical and operational barriers in precision-agriculture IoT applications also include cost, energy consumption, communication range, latency, storage, and processing limitations. These limitations indicate that the simple introduction of sensors, cameras, or robots is not sufficient. For digital agriculture to be viable in the field, solutions must jointly consider cost, energy autonomy, limited connectivity, local processing, and operational simplicity [4], [5].

Edge computing appears as an alternative for addressing some of these limitations, since architectures based exclusively on cloud computing may face critical problems in smart agriculture, including field communication latency, the need for low-power devices, and real-time decision constraints [6]. By shifting processing closer to the data sources, edge computing can enable low-latency responses and reduce network traffic in scenarios with limited connectivity. In this context, TinyML allows machine-learning inference to be executed on low-power embedded devices, enabling local detection without requiring continuous communication with cloud infrastructure.

Mobile robots have been developed for routine monitoring in row crops [7], however, they are used as the sole inspection agents, generally requiring complete or periodic patrols of the environment, including travel to locations where no relevant event has occurred. This repeated displacement can

This research was funded by Brazilian National Council for Scientific and Technological Development (CNPq), Brazil under Grant No. 307967/2022-0 ("Development of Intelligent Architecture for Applications on Internet of Robotic Things, Brazil"), No. 405852/2025-8 ("Intelligent and Autonomous 6G Networks: Data-Driven Design, Sizing, and Resource Allocation Using Artificial Intelligence, Brazil").

increase the operational cost and energy consumption of the operation, which is particularly problematic in agricultural systems already constrained by cost, energy autonomy and communication [4], [5]. At the same time, relying exclusively on fixed TinyML nodes may reduce communication and cloud-processing requirements, but their limited computational capacity and sensing conditions can restrict predictive performance [6]. The central issue is therefore not only whether a local TinyML classifier achieves high accuracy, but whether its predictions can be used as a reliable triage mechanism to reduce unnecessary robot activation while maintaining sufficient sensitivity to relevant agricultural events.

To investigate this issue, this work evaluates the visual monitoring of cashew leaves through an event-driven hierarchical architecture. The first layer consists of fixed low-power nodes based on the ESP32-S3, equipped with cameras and local TinyML inference for leaf detection and health classification. These nodes act as an initial screening layer. The second layer consists of a mobile robot that is dispatched only to locations where the fixed layer indicates a possible disease occurrence, enabling subsequent visual validation. Robot activation is defined by a confidence-threshold policy that regulates the trade-off among sensitivity to the diseased class, alert frequency, and operational economy.

The main contributions are:

- Demonstrate, through a controlled case study, how a fixed TinyML sensing layer can serve as a low-energy triage mechanism that activates a mobile robot only when local events are detected, enabling systematic evaluation of the trade-offs among predictive performance, alert frequency, and overall system energy consumption.
- Minimize global energy expenditure through the creation of an architecture where sensors trigger robotic validations only when there is local evidence of disease, replacing a portion of blind robotic patrolling;
- Develop a low-cost TinyML pipeline embedded on the ESP32-S3 for the visual analysis of cashew leaves, combining detection, cropping, and classification in quantized models.

II. Related Work

Recent works have explored TinyML for agricultural visual diagnosis on low-cost embedded devices. LeafSense [8] and the ESP32-CAM-based Early Blight detection system [9] demonstrate the feasibility of local plant disease inference, reducing cloud dependency, communication traffic, and latency. Other studies investigate mobile agents and distributed processing in rural environments, including UAV-assisted LoRa/LoRaWAN data collection [10], [11], UAV-supported TinyML for smart farming [12], and split inference for precision agriculture [13].

Hierarchical decision strategies have also been used to reduce computational cost in constrained platforms. Fouda et al. [14] proposed a confidence-based hierarchical AI model in which only selected samples are forwarded to a more expensive model. However, these works generally address TinyML diagnosis, mobile data collection, distributed inference, or hierarchical processing as separate components.

The gap explored in this work lies in the integration of these elements into an event-driven hierarchical architecture. The fixed TinyML node acts as a low-cost filter to decide when the robot should be triggered. The robot, in turn, stops patrolling all points blindly and instead validates only the locations flagged by the fixed layer. Thus, the central contribution lies not only in the accuracy of the embedded model or in robotic navigation, but in the analysis of the trade-off between predictive performance, the number of alerts, robotic displacement, and overall energy consumption.

III. Methodology

A. Proposed architecture and experimental setup

This work evaluates an event-driven hierarchical architecture to reduce the energy consumption of continuous agricultural monitoring. The proposed architecture consists of two levels. The first level comprises low-power fixed nodes based on the ESP32-S3, equipped with an OV2640 camera and LoRa P2P communication. These nodes locally execute a TinyML pipeline for the visual screening of cashew leaves. The second level consists of a TurtleBot3 Waffle Pi mobile robot, equipped with a Raspberry Pi 4 and a camera, triggered only when the fixed nodes detect signs of disease.

The mobile robot is dispatched to travel to the points flagged by the fixed layer, where it can perform visual inspections. In this evaluation, the modeled cost for the robotic level accounts for the travel to the alert locations and the return to the base, excluding any additional visual processing embedded on the robot.

The objective of the architecture is to replace blind robotic patrolling, in which the robot periodically traverses all points in the environment, with an on-demand policy. Thus, rather than being used as a continuous data collector, the robot acts as a mobile validation layer whenever an alert is issued by the fixed network. The baseline used for the energy comparison is the complete patrolling of the environment by the robot.

The evaluation was conducted using a hybrid experimental-simulated approach. The fixed level was evaluated experimentally on a test bench using the ESP32-S3, measuring the inference time and energy consumption per pipeline stage. The robotic level was evaluated in a Unity3D simulation integrated with ROS2, Nav2, and SLAM Toolbox. The simulated environment represents a controlled agricultural area with 25 plants arranged in a 5×5 grid, along with a fixed charging base for the robot. Each point on the grid was treated as a unit monitored by a fixed node, resulting in 25 local screening cycles per monitoring round.

Figure 1 represents the pipeline process from initial capture to the detection result for each individual leaf.

B. TinyML pipeline, data, and training

The pipeline executed on the fixed node begins with the capture of an original 1280×720 pixel image. To reduce information loss during resizing and to enable inference on the

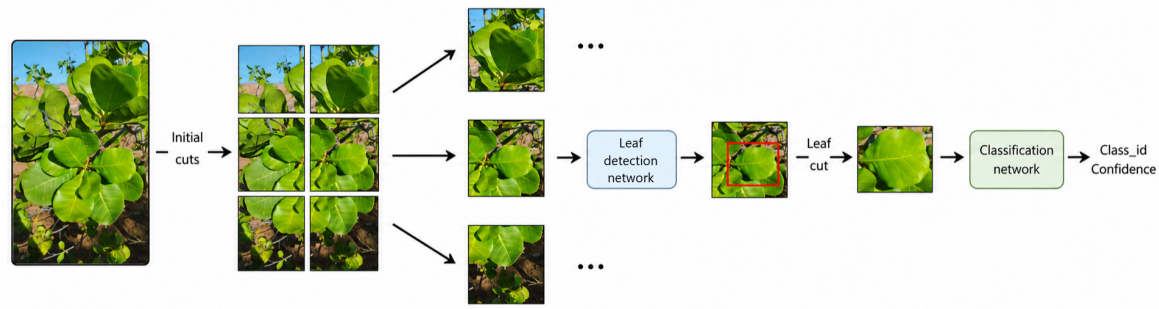


Fig. 1. Operational workflow of the solution up to leaf classification.

ESP32-S3, the image is divided into six partially overlapping crops. Each crop is resized to 256×256 pixels, preserving the aspect ratio through padding when necessary, and sent to the leaf detection network. The regions detected with a confidence greater than or equal to a threshold θ_d are cropped and resized for input into the classification network. Finally, if at least one leaf is classified as diseased with a confidence greater than or equal to θ_c, the node issues a LoRa P2P alert to request robot validation.

Two real cashew leaf datasets were used. The first contains 67 images with multiple leaves and was annotated with bounding boxes for training the detection network. The second was derived from the crops of the annotated leaves, separated into healthy and diseased classes, for training the classification network, totaling 143 diseased leaves and 558 healthy leaves. The models were trained using a 70/15/15 train/validation/test split, data augmentation only on the training set with the aim of increasing data variability and balancing classes, and early stopping based on validation loss. The classification was evaluated using stratified cross-validation with $k = 5$. Ultimately, the models were quantized to INT8 and exported in the .espdl format for execution with ESP-DL on the ESP32-S3.

The detection model receives 256×256×3 crops and outputs bounding boxes with confidence scores, with approximately 220k parameters. The classification model receives 96×96×3 leaf crops and outputs healthy/diseased predictions, with approximately 25k parameters. Both models were quantized to INT8 and deployed on the ESP32-S3 using ESP-DL.

The selection of thresholds was treated as an operational parameter of the architecture. A lower threshold tends to increase recall and the number of alerts, raising the robot's usage. A higher threshold tends to reduce activations but may increase false negatives. Therefore, the energy analysis was conducted based on the number of alerts per cycle, allowing the solution to be evaluated in scenarios ranging from sparse activity to the extreme case where all nodes request validation.

C. Energy modeling and robotic simulation

The energy cycle of the fixed node was modeled as the sequence: deep sleep, capture, detection, classification, LoRa transmission when there is an alert, and return to deep sleep. The measurements for capture, detection, and classification were performed using an FNIRSI FNB58 USB multimeter, with the ESP32-S3 operating at 240 MHz, Wi-Fi/Bluetooth disabled, and PSRAM enabled. Each stage was repeated 500 times, and the average energy was obtained by dividing the accumulated energy by the number of repetitions. The pre-processing and post-processing stages were incorporated into their corresponding functional blocks.

The fixed node's sleep term was incorporated into the energy model to represent the interval between successive monitoring cycles. Since the current in deep sleep mode is below the practical resolution of the USB multimeter used, this consumption was not measured directly on the test bench. Instead, the sleep power P_{sleep} was estimated from the electrical specifications of the ESP32-S3 in deep sleep mode [15]; thus, the power is obtained by:

$$P_{sleep} = 26.85\mu\text{W}$$

The sleep time T_{sleep} was defined based on the adopted operational scenario. A daily window of 10 hours with adequate lighting for the visual inspection of the plants and 10 monitoring cycles per day were considered. Thus, each node performs a local validation approximately every hour, yielding:

$$T_{sleep} = 1 \text{ h}$$

The energy cycle of the fixed node without an alert is given by:

$$E_{node_no_alert} = P_{sleep} \times T_{sleep} + E_{cap} + 6 \times E_{det} + C \times E_{cls}$$

$$E_{node_no_alert} = 26.85\mu\text{Wh} + E_{cap} + 6 \times E_{det} + C \times E_{cls}$$

Where E_{cap} is the capture energy, E_{det} is the processing energy for each initial crop, E_{cls} is the classification energy per cropped leaf, and C is the average number of leaves detected per original image. The cycle with an alert adds the cost of LoRa communication:

$$E_{node_alert} = E_{node_no_alert} + E_{LoRa}$$

The LoRa energy was measured considering the transmission of a compact alert packet, without sending an image. The adopted scenario uses a 20-byte payload, transmission power of 22 dBm, SF12, bandwidth of 125 kHz, coding rate of 4/5, CRC enabled, and a 1 s window for ACK without retries.

For the robot, the energy was estimated based on the declared autonomy of the TurtleBot3 Waffle Pi. It is important to note that since the same average robotic power is used for both policies, the comparison emphasizes the effect of reducing travel time rather than absolute robot power modeling. Considering a 19.98 Wh battery and an average autonomy of 2 hours [16], the average operating power was estimated at:

$$P_{robot} = \frac{19.98 \text{ Wh}}{2 \text{ h}} = 9.99 \text{ W}$$

The blind patrolling baseline was obtained by simulating the robot's visit to all 25 points on the grid. For the on-demand policy, a displacement matrix $T \in \mathbb{R}^{26\times26}$ (25 points and the base) was constructed, containing the simulated times between all pairs of points: base, plants, and return to base. This matrix was used to generate Monte Carlo scenarios with Y alerts per cycle. In each scenario, the active points were randomly drawn among the 25 grid cells, and the route was estimated by sequentially attending to the alerts with a final return to the base. The order of visitation was defined by a nearest-neighbor heuristic using the point-to-point time matrix, and the scenarios by quantity of points Y were repeated 1000 times.

The on-demand robotic energy is given by:

$$E_{robot}(Y) = P_{robot} \times T_{route}(Y)$$

The global energy consumption of the proposed architecture is the following:

$$\begin{aligned} E_{\text{global}}(Y) = Y \times E_{\text{node_alert}} \\ + (25 - Y) \times E_{\text{node_no_alert}} + E_{\text{robot}}(Y) \end{aligned} \quad (1)$$

In contrast, the consumption of blind patrolling is:

$$E_{blind} = P_{robot} \times T_{blind}$$

The percentage of energy reduction compared to the baseline is:

$$\text{Reduction}(Y) = 100 \times \left(1 - \frac{E_{global}(Y)}{E_{blind}}\right)$$

IV. Results

A. Performance of the models and the end-to-end pipeline

The leaf detection network achieved a precision of 81.18%, a recall of 70.2%, an mAP_{50} of 80.47%, and an mAP_{50-95} of 48.4% on the test set. Although the mAP_{50-95} indicates a limitation in the geometric precision of the bounding boxes, this impact is mitigated by the intermediate role of detection in the pipeline, since the bounding boxes are used to generate the crops that will be evaluated by the classification network.

In the confidence threshold analysis for detection, the best balance between precision and recall occurred at θ_d = 30%, with an F1-score of 75.31%, a precision of 81.97%, a recall of 69.65%, and 1.20 false positives per image. As expected, higher thresholds increased precision and reduced false positives, but also decreased recall.

The classification network showed consistent average performance in stratified cross-validation. The average accuracy was 95.72 ± 2.02%, with a precision of 93.71 ± 3.98%, a recall of 93.30 ± 2.84%, and an F1-score of 93.47 ± 2.93%. The average false negative rate for the diseased class was 10.49 ± 4.92%, indicating that, although the rate is low, the critical class presents relevant variation among folds.

The end-to-end evaluation considered the complete chain: detection, leaf cropping, classification, and final decision. A conditional threshold (θ_c) applied only to the diseased class was also evaluated. The adopted rule considers a prediction as an alert only when the predicted class is diseased and the confidence is greater than or equal to 0.63. This post-processing was adopted because a significant portion of the false positives for the diseased class was concentrated in low-confidence predictions.

For this, 10 original images were used (all images from the test set), 4 of which were composed only of healthy leaves and 6 containing at least one diseased leaf. The images produced 266 detected leaf crops. Table I summarizes the end-to-end robot activations, and Table II summarizes the performance of the detection and classification models, as well as the end-to-end pipeline.

TABLE I
ACTIVATIONS PER DECISION RULE

Decision rule	Expected alerts	True alerts	False alerts	Missed alerts	Total activations
$\theta_c = 0.50$	6	6	2	0	8
Diseased $\theta_c \geq 0.63$	6	6	1	0	7

The end-to-end accuracy was calculated considering the 266 crops produced by the detection stage. The total pipeline errors include both classification errors on valid crops and errors associated with invalid or inconsistent crops generated by the detection stage. Thus, the 27 classification errors observed with $\theta_c = 0.50$ represent only the errors on the 255 valid crops, while the 38 total errors also incorporate the failures attributed to the detection stage. With the application of the conditional threshold for the diseased class, defined as $\theta_{c_diseased} \geq 0.63$, the number of classification errors dropped to 18.

However, since the robot trigger is decided per original image, rather than per individual crop, the most relevant metric for the architecture is the occurrence of at least one positive prediction for disease within the image. Since 6 of the 10 images contained diseased leaves, the expected behavior would be the generation of 6 true alerts. With $\theta_c = 0.50$, all images with the presence of disease generated an alert, but 2 images composed entirely of healthy leaves also produced false alerts

TABLE II
GENERAL ASSESSMENT

Assessment	Result
Detection - precision	81.18%
Detection - recall	70.20%
Detection - mAP_{50}	80.47%
Detection - best F1	75.31%
Classification - average accuracy	95.72 ± 2.02%
Classification - average F1	93.47 ± 2.93%
End-to-end ($\theta_c = 0.50$)	85.71%
End-to-end (Diseased $\theta_c \geq 0.63$)	89.10%
Total pipeline errors ($\theta_c = 0.50 \rightarrow$ Diseased $\theta_c \geq 0.63$)	$38 \rightarrow 29$
Classification errors ($\theta_c = 0.50 \rightarrow$ Diseased $\theta_c \geq 0.63$)	$27 \rightarrow 18$
Diseased false positives ($\theta_c = 0.50 \rightarrow$ Diseased $\theta_c \geq 0.63$)	$20 \rightarrow 9$
Diseased false negatives ($\theta_c = 0.50 \rightarrow$ Diseased $\theta_c \geq 0.63$)	$7 \rightarrow 9$

due to false positives from the diseased class. Thus, this threshold resulted in 8 robot activations in total: 6 true and 2 false.

With the conditional threshold $\theta_{c_diseased} \geq 0.63$, all 6 images with the presence of disease continued to generate an alert, while the number of healthy images producing a false alert dropped from 2 to 1. Therefore, the total number of robot activations was reduced from 8 to 7, without loss of sensitivity at the original image level. Although false negatives occurred in individual crops of diseased leaves, these errors did not prevent the robot from being triggered, as other leaves or crops from the same image were still correctly classified as diseased.

The conditional θ_c increased the end-to-end accuracy from 85.71% to 89.10%, with an absolute gain of 3.39 percentage points. The main effect was the reduction of false positives for the diseased class, from 20 to 9 cases. Conversely, false negatives increased from 7 to 9 cases. Therefore, at the individual crop level, the conditional threshold reduces false positives for the diseased class but slightly increases false negatives. At the operational level of the original image, however, this increase did not result in missed alerts in the evaluated set, since all images with the presence of disease continued to produce at least one positive prediction.

B. Quantization, timing, and energy on the ESP32-S3

INT8 quantization moderately reduced the performance of the classification network compared to the FP32 model. In the fold used for comparison, the F1-score dropped from 96.61% to 93.03%, while the accuracy fell from 97.86% to 95.71%. This loss is consistent with the reduction in numerical precision caused by quantization and was considered acceptable given the need for efficient execution on the ESP32-S3.

On the embedded hardware, the detection network presented an average time of 175.95 ms per crop, while classification presented an average time of 35.76 ms per leaf. Considering 6 initial crops per image and an average of 24.18 leaves per original image in the validation split, the complete cycle of the fixed node presented an average time of 1.987 s without an alert. When there is an alert, the total time rises to 4.334 s, primarily due to the LoRa transmission and the ACK window.

Table III details the times per process in the pipeline.

TABLE III
SYSTEM TIME AND ENERGY METRICS

Metric	Value
T_{cap}	66.66 ms
T_{det} per cut	175.95 ms
T_{cls} per leaf	35.76 ms
Mean leaves per image (C)	24.18
T_{LoRa}	2.347 s
No alert cycle	1.987 s
Alert cycle	4.334 s
E_{cap}	7.33 μWh
E_{det} per cut	23.625 μWh
E_{cls} per leaf	4.6 μWh
E_{LoRa}	0.3226 mWh
$E_{node_no_alert}$	0.2872 mWh
E_{node_alert}	0.6098 mWh

The results show that the energy cost of the fixed node is low compared to the displacement cost of the robot. Even in the cycle with an alert, the node's consumption remains below 0.61 mWh. This supports the main objective of the architecture: utilizing a low-power fixed network helps to avoid energetically expensive robotic displacements.

C. Energy comparison between blind patrolling and on-demand validation

The blind patrolling baseline was obtained by applying the same nearest-neighbor routing heuristic used in the on-demand policy, with a total time of 414.81 s. Considering the estimated average power of 9.99 W, the energy consumption of the complete patrolling was:

$$E_{blind} = 9.99 \text{ W} \times 0.1152 \text{ h} = 1.151 \text{ Wh}$$

In the on-demand policy, the global energy includes both the fixed network and the robot's displacement to the points that issued an alert. Table IV presents the results for different numbers of alerts per cycle.

TABLE IV
ENERGY CONSUMPTION AND REDUCTION ANALYSIS

Alerts per cycle (Y)	Level 1 Energy	Level 2 Energy	Global Energy	Reduction vs. blind
1	7.502 mWh	185.82 mWh	193.32 mWh	83.20%
5	8.792 mWh	397.61 mWh	406.39 mWh	64.69%
10	10.405 mWh	584.42 mWh	594.82 mWh	48.32%
20	13.631 mWh	958.04 mWh	971.67 mWh	15.58%
25	15.244 mWh	1,151 mWh	1,166 mWh	-1.31%
Blind patrolling	—	1,151 mWh	1,151 mWh	—

The results indicate that the on-demand architecture remains energetically advantageous as long as the number of alerts does not approach a complete inspection. With only 1 node alert

per cycle, the overall reduction is 83.20%. Even with 20 node alerts, equivalent to 80% of the grid nodes, there is still a 15.58% savings compared to blind patrolling. At the limit of 25 node alerts, the proposal converges to a complete sweep but incurs the additional cost of the fixed network, resulting in an increase of only 1.31% in consumption.

The analysis highlights that the energy gain of the architecture does not stem from an isolated reduction in computational cost, but rather from substituting mechanical energy with low-power local inference. Level 1 consumes a few milliwatt-hours per cycle, whereas robotic displacement consumes hundreds of milliwatt-hours even in scenarios with few points.

The trigger events observed in the 10 test images were not used as a direct estimate of field prevalence, as the set was defined to evaluate the behavior of the pipeline and contains a controlled proportion of images with disease. For this reason, the energy analysis was parameterized by Y alerts per cycle. Nevertheless, the end-to-end results show that the conditional threshold reduces unnecessary trigger events at the original image level, which directly corresponds to the variable Y used in the energy modeling.

D. Assessment limitations

The results should be interpreted as a hybrid experimental-simulated validation. The energy of the ESP32-S3 was physically measured on a test bench, whereas the robot's energy was estimated based on the declared autonomy of the TurtleBot3 Waffle Pi. Furthermore, the Unity/ROS2 simulation captures path planning and navigation time in the controlled environment, but it does not replace testing with a physical robot in real field conditions.

The dataset also presents limitations in size and variability, especially because the crops used in the classification are derived from the detection dataset. Therefore, generalization to other environments, lighting conditions, disease stages, and planting scales must be evaluated in future works with larger datasets and a more rigorous separation by plant or source image. Even with these restrictions, the results demonstrate the energetic viability of the proposed architecture and show that local screening on the ESP32-S3 can substantially reduce the use of the mobile robotic layer.

V. Conclusion

This work evaluated an event-driven hierarchical architecture for visual agricultural monitoring, combining ESP32-S3 TinyML fixed nodes with an on-demand mobile robotic layer. The results indicate that local visual screening can serve as a low-cost energy filter for reducing blind robotic patrolling. The detection model achieved 80.47% mAP_{50}, while the classification model reached 95.72% average accuracy and 93.47% average F1-score. On the ESP32-S3, the complete fixed-node cycle consumed less than 0.61 mWh even with an alert.

The conditional diseased-class threshold increased end-to-end accuracy from 85.71% to 89.10% and reduced diseased false positives from 20 to 9, decreasing unnecessary robot activations in the evaluated set. In the energy analysis, the proposed architecture reduced consumption by 83.20% with 1 alert per cycle and by 15.58% with 20 alerts. Only when all 25 nodes generated alerts did the approach become slightly less efficient than blind patrolling.

These results provide preliminary evidence that substituting part of the robotic inspection effort with low-power TinyML screening can improve the energy-efficiency of monitoring.

As for limitations, the evaluation combines real measurements on the ESP32-S3 with robotic simulation, and the dataset is still restricted. Future works should validate the architecture with a physical robot in the field, expand the image dataset, and investigate dynamic strategies for adjusting thresholds according to disease prevalence, energy cost, and the risk of false negatives.

References

[1] S. Vijayakumar, V. Murugaiyan, S. Ilakkiya, V. Kumar, R. M. Sundaram, and R. M. Kumar, "Opportunities, challenges, and interventions for agriculture 4.0 adoption," *Discover Food*, vol. 5, 2025.

[2] V. K. Quy, N. V. Hau, D. V. Anh, N. M. Quy, N. T. Ban, S. Lanza, G. Randazzo, and A. Muzirafuti, "IoT-enabled smart agriculture: Architecture, applications, and challenges," *Applied Sciences*, vol. 12, no. 7, 2022.

[3] R. Bezerra, S. Tadokoro, and K. Ohno, "Ai-iot-robotics integration: Survey of frameworks, emerging trends, and the path toward connected robotics," *IEEE Internet of Things Journal*, vol. 13, no. 10, pp. 20398–20412, 2026.

[4] G. S. Hundal, C. M. Laux, D. Buckmaster, M. J. Sutton, and M. Langemeier, "Exploring barriers to the adoption of internet of things-based precision agriculture practices," *Agriculture*, vol. 13, no. 1, 2023.

[5] S. Mansoor, S. Iqbal, S. M. Popescu, S. L. Kim, Y. S. Chung, and J.-H. Baek, "Integration of smart sensors and IoT in precision agriculture: trends, challenges and future prospectives," *Frontiers in Plant Science*, vol. 16, 2025.

[6] R. Gong, H. Zhang, G. Li, and J. He, "Edge computing-enabled smart agriculture: Technical architectures, practical evolution, and bottleneck breakthroughs," *Sensors*, vol. 25, 2025.

[7] W. S. Barbosa, A. I. S. Oliveira, G. B. P. Barbosa, A. C. Leite, K. T. Figueiredo, M. M. B. R. Vellasco, and W. Caarls, "Design and development of an autonomous mobile robot for inspection of soy and cotton crops," in *2019 12th International Conference on Developments in eSystems Engineering (DeSE)*, pp. 557–562, IEEE, 2019.

[8] R. Samanta, B. Saha, and S. K. Ghosh, "Leafsense: A portable, low-cost, low-power plant disease diagnostic device using TinyML," in *4th International Conference on AI-ML Systems (AIMLSystems 2024)*, 2024.

[9] A. Bobrek, "IoT-enabled early blight detection system for tomatoes using ESP32-CAM," *International Journal of Modern Engineering Research (IJMER)*, vol. 15, no. 6, pp. 52–59, 2025.

[10] A. Andreadis, G. Giambene, and R. Zambon, "Design and analysis of a LoRa-based system with scheduled transmissions from IoT nodes to UAV in rural areas," *Ad Hoc Networks*, vol. 175, 2025.

[11] L. Holtorf, I. Titov, F. Daschner, and M. Gerken, "UAV-based wireless data collection from underground sensor nodes for precision agriculture," *AgriEngineering*, vol. 5, pp. 338–354, 2023.

[12] A. M. Hayajneh, S. A. Aldalahmeh, F. Alasali, H. Al-Obiedollah, S. A. Zaidi, and D. McLernon, "Tiny machine learning on the edge: A framework for transfer learning empowered unmanned aerial vehicle assisted smart farming," *IET Smart Cities*, vol. 6, pp. 10–26, 2024.

[13] I. Ali, N. Bovee, S. Bitla, G. Patapanchala, R. Gangoni, and S.-S. Ho, "Split inference for precision agriculture," in *IEEE International Conference on Data Mining Workshops (ICDMW)*, 2025.

[14] M. M. Fouda, S. Sakib, Z. M. Fadlullah, N. Nasser, and M. Guizani, "A lightweight hierarchical AI model for UAV-enabled edge computing with forest-fire detection use-case," *IEEE Access*, 2022.

[15] Espressif Systems, *ESP32-S3 Series Datasheet*, oct 2024. ver. 2.1.

[16] ROBOTIS, "Turtlebot3: Features." https://docs.robotis.com/docs/systems/turtlebot3/features. Accessed: June 15, 2026.